

Algoritmalar ve Veri Yapıları Bootcamp Ödev 2.

1. Aşağıda verilen her biri iç içe (nested) ikişer adet döngü yapısı içeren üç farklı kısa algoritma parçası için ayrı ayrı, zaman karmaşıklık ifadelerinin (algoritmaların temel operasyonlarının koşu zamanında toplam kaç kez çalıştırılacağına dair ifadeler) asimptotik olarak ne olduğunu **big-Oh** notasyonu ile tespit edip belirtiniz. Algoritmaların her biri için temel operasyonun (sum+1 ifadesindeki) toplama olduğunu kabul edebilirsiniz.

a.

```
sum ← 0
for j ← n downto 1 step j/2 do
  for i ← 0 to j-1 do
    sum ← sum + 1
```

b.

```
sum ← 0
for i ← 1 to n-1 step i*2 do
  for j ← 0 to i-1 do
    sum ← sum + 1
```

c.

```
sum ← 0
for i ← 1 to n-1 step i*2 do
  for j ← 0 to n-1 do
    sum ← sum + 1
```

2. Aşağıda verilen fonksiyon çiftlerinin ($f(n)$ ve $g(n)$) her biri için, aralarındaki “order-of-growth” ilişkisinin ne olduğunu bulmaya ve şu soruları cevaplamaya çalışın:

$f(n)$ mi $O(g(n))$ yani f 'in büyüme hızı g 'ye göre daha mı yavaş, ya da başka bir deyişle n sonsuza doğru gittikçe g fonksiyonu f 'e göre daha çabuk mu büyüyor;

yoksa:

$g(n)$ mi $O(f(n))$ yani g 'in büyüme hızı f 'e göre daha mı yavaş, ya da başka bir deyişle n sonsuza doğru gittiğinde f fonksiyonu g 'ye göre daha çabuk mu büyüyor;

Ya da hem **$f(n)$ is $O(g(n))$** , hem de **$g(n)$ is $O(f(n))$** diyebilir miyiz, yani n sonsuza gittiğinde her iki fonksiyon da birbirleriyle aynı büyüme hızını gösteriyorlar ve dolayısıyla “order-of-growth” karakterleri birbirleriyle aynı?

Yukarıdaki sorulara verdiğiniz cevaplara göre aşağıdaki tabloyu doldurabilirsiniz.

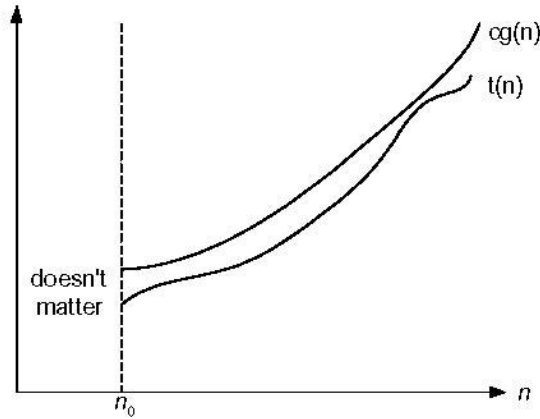
	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$f(n) = n(n+1)$ $g(n) = 2000n^2$		
$f(n) = 100n^2$ $g(n) = 0,01 n^3$		
$f(n) = (n-1)!$ $g(n) = n!$		
$f(n) = \log^2 n$ $g(n) = \log n^2$		
$f(n) = 2^n$ $g(n) = 2^{n-1}$		
$f(n) = \log n$ (Logarithm base 2) $g(n) = \ln n$ (Logarithm base e)		

Yukarıdaki sorulara cevap verirken ve tabloyu doldururken;

- Ya order-of-growth (big-Oh) kurallarının tanımından gelen ilgili eşitsizliklerin geçerli olmasını sağlayacak uygun n_0 , ve c sabitlerini bulun, yani;

$f(n)$ 'in $O(g(n))$ olabilmesi için aşağıdaki koşulu sağlayan pozitif bir c sabiti ve negatif olmayan bir n_0 tamsayısı olmalı, şöyle ki:

$$f(n) \leq c g(n) \text{ her bir } n \geq n_0 \text{ değeri için}$$



- Ya da, fonksiyonları yeterli miktarda n değeri için hesaplayarak, koordinat sisteminde eğrilerini çizerek ya da hesapladığınız değerleri bir tabloya doldurarak, n sonsuza doğru ilerlediğinde her iki fonksiyonun “order-of-growth” ilişkilerinin ne olacağına dair bir tahminde bulunmaya çalışabilirsiniz.
- Son olarak verilen iki fonksiyon arasındaki “order-of-growth” ilişkisinin tespitinde kullanılabilecek bir diğer yöntem de, Stirling formülü olarak bilinen formülü kullanmaktır. O da şunu söyler:

Eğer $\lim f(n)/g(n) = 0$ ise: **$f(n) O(g(n))$ dir.**

Eğer $\lim f(n)/g(n) = c > 0$ ise: **hem $f(n) O(g(n))$ hem de $g(n) O(f(n))$ ’dir**

Eğer $\lim f(n)/g(n) = \infty$ ise: **sadece $g(n) O(f(n))$ ’dir.**